

ABLATION STUDY

This menu is our Phase 3 experiment plan. We keep the main goal fixed: virtual graph → GraphSAGE → 64-d embedding → same evaluation. Then we change one part at a time to see what helps.

A tests how GraphSAGE learns: A1 uses random-walk context pairs, closer to old I2V; A2 uses direct virtual edges, cleaner and simpler.

B tests how a node combines neighbor information: mean is standard GraphSAGE; weighted mean uses virtual-edge similarity; sum/max are small checks.

C tests depth: 1 layer sees immediate neighbors, 2 layers sees neighbors-of-neighbors, 3 layers may over-smooth.

D tests features: structural-4 gives useful graph signals; degree-only checks minimal information; random checks whether graph alone is enough.

E tests the real research question: which virtual graph works best, Ψ , degree, or centrality, and which K.

Design reason: fair ablation. Change one thing, compare results, understand why. This prevents confused conclusions and wasted experiments.

Think of Phase 3 like a controlled experiment. You will build one basic GraphSAGE system first, then change only one part at a time.

A. Positive-pair design means: how does the unsupervised GNN know which nodes should become close in embedding space?

A1 = walk co-occurrence positives. You run random walks on the virtual graph. If node A and node B appear near each other in the walk window, they become a positive pair. This is closer to old I2V/Skipgram because old I2V also learned from walk context.

A2 = direct virtual-edge positives. No walks. If node A is connected to node B in the virtual graph, they are a positive pair. This is simpler and cleaner for GNN because the virtual graph already says “these two nodes are structurally similar.”

In both A1 and A2, you also sample negative pairs: random nodes that should not be close. The loss says: pull positive pairs together, push negative pairs apart.

B. Aggregation means: when GraphSAGE updates a node, how does it combine information from its neighbors?

Mean GraphSAGE: take average of neighbor features. This is the standard, safest first version.

Weighted mean: use virtual edge weights. If edge weight is high, that neighbor contributes more. This is important because Phase 2 DeepWalk ignored weights, but GNN can use them.

Sum: adds neighbor information. It may emphasize high-degree nodes too much.

Max: keeps the strongest feature value from neighbors. It is just an ablation, not the main choice.

C. Depth means: how many message-passing layers?

1 layer: node sees direct virtual neighbors only.

2 layers: node sees virtual neighbors and their neighbors. This is usually a good default.

3 layers: sees farther, but may over-smooth. Over-smoothing means many node embeddings become too similar.

Because our virtual graph already connects structurally similar nodes directly, 2 layers is likely enough.

D. Features means: what information each node starts with before message passing.

Structural-4:

degree, eigenvector centrality, psi score, clustering coefficient

This is our best default because it gives GraphSAGE structural information.

Degree-only: checks whether just degree is enough.

Random features: checks whether the graph structure alone is enough. If random features still work well, then the virtual graph itself is doing most of the work.

“Change one thing, compare results” means this: keep everything fixed, change only one setting, then compare F1/AUC.

Example:

Same		dataset	=			Cora
Same	graph		=		psi,	K=10
Same	model		=	2-layer		GraphSAGE
Same	features		=			structural-4
Same	seeds		=			42, 43, 44

Only
change:
A1 loss vs A2 loss

Then you can honestly say: the result changed because of the positive-pair design, not because of another hidden difference.

Implementation order:

Step 1: Build one working `encoder.py`.

Start with:

GraphSAGE				layers
2				aggregation
mean				features
structural-4				graph
psi		virtual		
K=10				
A2		direct-edge		positives
64-d embeddings				

Why A2 first? It is simpler: no walks, fewer moving parts, more natural for GNN.

Step 2: Train it and save embeddings:

```
output/cora/k10/gnn_sage_psi_A2_s42.emb
```

Run existing evaluation:

```
eval_nodeclass.py → F1
eval_linkpred.py → AUC
```

Save results:

```
results/phase3/
```

Step 3: Add A1, do not delete A2.

Run same setup again, but change only positive-pair mode:

```
A2 direct edges
vs
A1 walk co-occurrence
```

Then compare. Whichever is better becomes our default training objective.

Step 4: After A winner is chosen, test B.

Keep everything fixed, change only aggregation:

```
mean
weighted mean
sum
max
```

Choose the best/safest one.

Step 5: Then test C depth.

Keep A and B fixed, change:

```
1 layer
2 layers
3 layers
```

Choose best depth.

Step 6: Then test D features.

Keep A, B, C fixed, change:

```
structural-4
degree-only
random
```

This tells you whether features matter or graph structure matters more.

Step 7: Then do the real virtual-graph study.

Use the chosen GraphSAGE setup and test:

psi vs degree vs centrality
K = 5, 10, 20

This is the our main question: which virtual graph is best for which task/data?